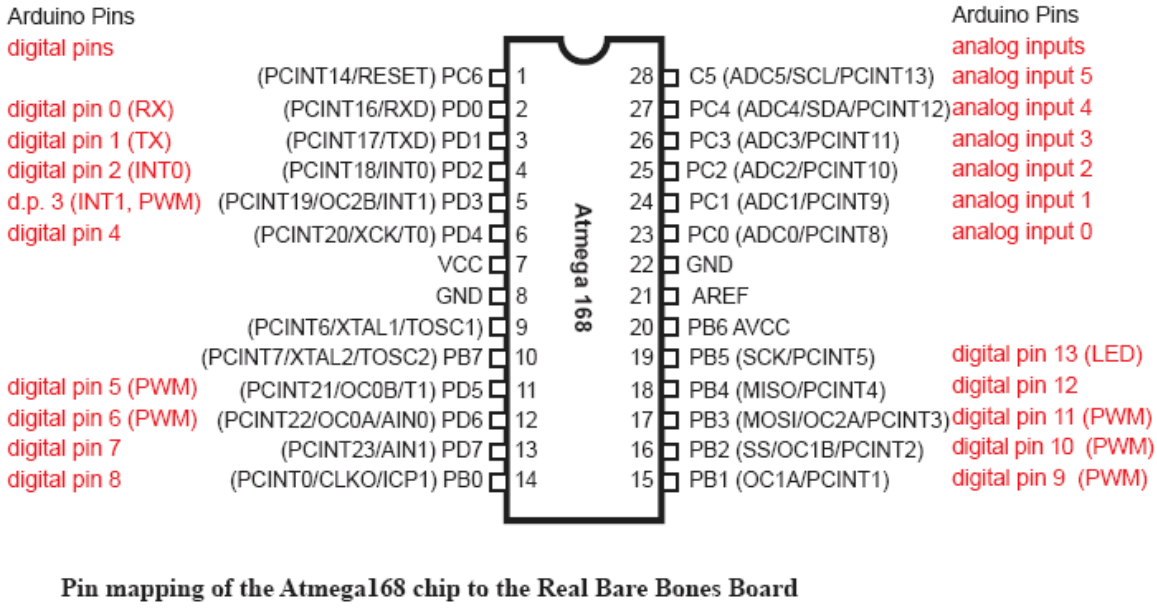
ECE 3218 Lab #5 - Ports, Interrupts, and (Bouncy) Switches

Background

The previous two labs made use of ports for input / output functions. Recall that port pins can have other functions as well depending on how the processor's various control registers are configured. All the port pins can be configured to generate interrupts on a logic sense change (low-to-high or high-to-low). Also Port D pins 2 and 3 can be configured to generate interrupts in response to four different input signal attributes. In this lab we will set up ports to provide specific values on their pins and, simultaneously, to generate interrupts in response to both external and internal events.



Pin mapping of the Atmega168 chip to the Real Bare Bones Board

Figure 1. Pin mapping of the ATMEGA 168/328P DIP processor chip to the headers on the Arduino Uno Board.

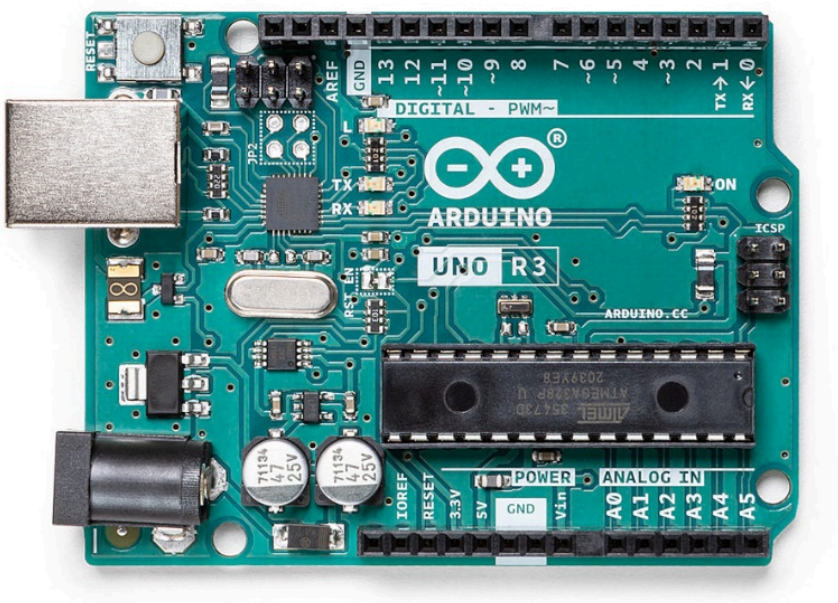


Figure 2. The Arduino Uno Board shows the header sockets for Port D pins 0-7 (digital pins 0-7), Port B pins 0-5 (digital pins 8-13) at the top, and Port C pins 0-5 (analog input pins 0-5) at the bottom. At the bottom center there is an 8-socket header for various power, ground, and reset connections.

Before The Lab

Part 1

The following is a program that will generate an interrupt in response to the push of a momentary button switch attached to the INT0 pin. The ISR toggles the power to an LED. In theory the first time the button is pushed the LED will turn on. The next time the button is pushed it will turn off. The reality is somewhat different.

Enter the following assembly program into a project or solution in Microchip Studio:

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
; Lab 5: Interrupts and (bouncy) switches
;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;


.DEF ZERO=R0


.dseg            ; SRAM variables
LEDON: .byte 1   ; LED status ~0 ==> ON, 0 ==> OFF
PCLH: .byte 1    ; Port C pin 0 status 1 ==> High, 0 ==> Low for PCINT0
COUNT: .byte 4   ; number of times the interrupt routine was executed


; interrupt table
; this program requires RESET, INT0, and PCINT1
.cseg
   jmp    RESET    ; Reset Handler
   jmp    EXT_INT0 ; IRQ0 Handler (external hardware interrupt)
;.org PCI1addr     ; This is used in part 4, the timed lights, comment it out here
;  jmp    PC_INT1  ; PCINT1 Handler (pin change interrupt on Port B) for part 4


RESET:
main:
; initialize stack pointer
   ldi    r16,high(RAMEND)
   out    sph,r16
   ldi    r16,low(RAMEND)
   out    spl,r16
; (R0)<-0x00 aliased to ZERO
   clr    ZERO
; go to program
   rcall  L06P01

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
; Lab 5 program 1: Toggle LED with momentary push-button switch
;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
L06P01:
; initialize variables
; LED status flag
   sts    LEDON,ZERO ; 0 ==> LED is not on
   sts    LEDON+1,ZERO ; clear this byte to make the next 4 easier to see

; number of times the ISR has been executed
   sts    COUNT,ZERO ; clear ISR count high byte
   sts    COUNT+1,ZERO ; clear ISR count upper middle byte
   sts    COUNT+2,ZERO ; clear ISR count lower middle byte
   sts    COUNT+3,ZERO ; clear ISR count low byte


; set up port B for output to LED
   sbi    DDRB,0 ; bit 0 in Port B DDR <- 1, data direction out
   cbi    PORTB,0 ; bit 0 in Port B <- 0, turn off the LED


; set up hardware external interrupt INT0 to trigger when the Arduino board
; digital pin 2 (aliased to both INT0 and Port D pin 2) is pulled low
```

```
    ; set up Port D pin 2 for input with pull-up to logic level high
       cbi    DDRD,2              ; (a) set it to input
       sbi    PORTD,2             ; (b) engage the 5v pull-up


    ; Now set up for a hardware interrupt on INT0 (momentary switch)
        ; INT0 is on the same pin (Port D pin 2)
       sts    EICRA,r0 ; clear bits 0,1 so that IRQ is generated when pin is pulled low
       sbi    EIMSK,0       ; enable hardware interrupt INT0; bit 0 <- 1;
       sei            ; enable interrupts globally

; wait here for interrupt (Hit break, ||, in debug mode to pause execution here)
    SPIN:
       rjmp SPIN




    ; ISR for external interrupt 0 (INT0)
    EXT_INT0:


    ; load count of number of times ISR has been executed and increment it
       lds r27,COUNT ; load the current ISR count, high byte
       lds r26,COUNT+1 ; load the current ISR count, upper middle byte
       lds r25,COUNT+2 ; load the current ISR count, lower middle byte
       lds r24,COUNT+3 ; load the current ISR count, low byte
       adiw r25:r24,1 ; increment the count
       brcc NOUHINC ; if C==0, nothing to add to the upper half
       adiw r27:r26,1 ; else add one to upper half
    NOUHINC: ;


    ; toggle the LED state
       lds r17,LEDON ; get the LED status in r17
       tst r17 ; is it zero?
       brne ITSON ; no? then the LED is on; go turn it off
       sbi PORTB,0 ; yes? then it's off; turn it on
       sbr r17,1 ; set the flag in r17
       rjmp DUNIT
    ITSON:
       cbi PORTB,0 ; turn it off
       clr r17 ; clear the flag in r17
    DUNIT:
       sts LEDON,r17 ; save r17 to the LED status flag


    ; save the ISR execution count
       sts COUNT,r27 ; update the ISR count, high byte
       sts COUNT+1,r26 ; update the ISR count, upper middle byte
       sts COUNT+2,r25 ; update the ISR count, lower middle byte
       sts COUNT+3,r24 ; update the ISR count, low byte

       reti
```

Analyze the program logic by reading through it carefully. Describe what each block of code does. Study it until you thoroughly understand what the program does. Test the program with the simulator. In the lab you will build a simple circuit that connects to the Uno boards and at

once controls the program flow and is controlled by it. From this experiment you will observe the phenomenon of *Switch Bounce.*

Part 2

Also enter and simulate the following code for a subroutine that implements a millisecond timer on the ATmega328P.

```
; 1 msec timer subroutine (exact; time includes call and return)
; wait1ms expends 16000 clock cycles.
; rcall wait1ms uses 3 clock cycles
; Therefore from the rcall in the calling program through the
; ret in the subroutine wait1ms expends
; 3+1+1+1+(3996*4)[in loop]+3[end of loop]+1+1+1+4 = 16000cc
; which at 16MHz (cc/s) equals 1ms.
wait1ms:
    ldi zh,0x0F ; 1cc
    ldi zl,0x9D ; 1cc
    clz ; 1cc
waitloop: ; countdown 0x0F9D = 3997dec
    sbiw z,1 ; 2cc
    brne waitloop ; 2cc true, 1cc false
    nop ; 1cc
    nop ; 1cc
    nop ; 1cc
    ret ; 4cc
```

During Lab you will implement a very simple modification to the ISR that will overcome the switch bounce problem. To do that you will need an n-millisecond timer. In advance of the lab, write and test a subroutine that will wait for n milliseconds before returning. Have your subroutine do that by repeatedly calling the wait1ms subroutine. How wide in bytes does the counter need to be to increment up to 1000?

Part 4:

Copy the program from part 1 and modify it so that a pin change interrupt is generated on Port C Pin 0 (aka PCINT8 or analog input 0) once per second. Each second it will toggle two LEDs so that one is on and the other is off alternately. One LED will be controlled by the value on PortC0. The other will be toggled by the ISR similarly to the ISR in program 1. See part 4 in the next section for advice on how to make that happen.

All written programs should be tested with Simulator before the lab. Remember to use the I/O window to verify hardware output

In Your Notebook

Sketch the schematic that you will be building. You will need LEDs, resistors, and switches. You will use this to help you wire up the breadboard when you are in the lab. It is helpful to include both the ATMEGA328P port names and the Uno pin numbers on your schematic.

During the Lab

Part 1:

Wire the circuit on the test board based on your schematic sketch. Note that one side of the pushbutton switch is already connected to ground.

Part 2:

After wiring the circuit connect the Arduino board to the Atmel-ICE and both of them via USB to your computer. Assemble the first program, download it and run it. After it has run for a few seconds, hit the break-all button, ||, in Microchip Studio (equivalently, hit <ctrl>-F5 on your keyboard). The yellow arrow should appear on the "SPIN" loop. Look at data IRAM and verify that locations 0x0100–0x0105 are all zero.

Continue the program by hitting the green arrow (or F5). Observe the LED that is connected to digital pin 8 while you press the button switch firmly, hold it for about a half second, and release it. What did the LED do? Now <ctrl>-F5 or hit the break-all button. Find the 32-bit hex number in IRAM 0x0102-0x0105. That number is COUNT in the program. What does it represent? You pushed the button once; why is the number so large? Run the program that way 4 more times and take the average of the 5 numbers.

How much time does it take for the ISR to run? To figure that out, write down the number of cycles per instruction for each instruction in the ISR, including the interrupt call and return, then sum them. That's the amount of time spent in the ISR. Multiply that by the average of your five counts then by (1/16,000,000) s. To get a more accurate estimate of the expected value of the time elapsed during switch bounce you would need to know the expected number of times the RJMP SPIN instruction is executed. How could one modify the code to determine the average time between bounces?

Part 3:

Usually switch bounce[1] is compensated for by a de-bounce circuit that uses an SR-Latch[2] or a Schmitt trigger circuit[3]. A microcontroller can be programmed to wait for the bouncing to stop. That is what you are going to do here.

Put a time delay at the beginning of the ISR. It should be long enough so that bouncing ends before the timer expires. Use the n msec delay program you wrote in part 3 of the pre-lab. Determine how long a delay is needed to enable your circuit to work as intended – push the button once to turn the LED on, push it again to turn it off. Use the time you calculated above as a first guess.

When the program is working correctly, COUNT will equal the number of times that you actually pushed the button and toggled the LED. Verify that by pushing the button 10 times, breaking execution and looking at the count variable in memory.

Part 4:

Now Test your second program. It uses the pin change interrupt on Port C Pin 0 to toggle the same LED as in part 1 on and off at 1Hz. The other LED, connected to Port C Pin 0, should be on when the other is off and vice versa. You can write program 2 by modifying a copy of your first program. Set up Port B as before. This program will use Port C Pin 0 rather than Port D for the interrupt that triggers the LED toggle. Set up Port C Pin 0 for output and set it to logic high. Now set up pin change interrupt PCINT1. This also occurs on Port C. Enable PCINT8. That pin coincides with Port C Pin 0. Your main program should contain an endless loop that counts down for 1 second then toggles the output on PORTC0. That is going to cause a pin change interrupt. You must write an ISR for PCINT1 that checks the status of the LED on PORTB0 (using the status variable as before), toggles it accordingly, and saves the status. Although similar to the interrupt handler in part one, this ISR requires no timers. Why?

After the Lab

Make listings of your corrected, annotated programs. Note the number of times the ISR was called in part 1 for a single button push and note the delay time necessary for the bouncing to stop. In the second program, we used output to a port to trigger an interrupt on that port. This seems a bit odd. Since the program knows when it is going to change the output, it could simply call a subroutine to toggle the light rather than setting up an interrupt. Find one or more applications or situations where triggering an interrupt this way makes sense.

1. http://en.wikipedia.org/wiki/Switch_bounce#Contact_bounce
2. http://en.wikipedia.org/wiki/Latch_(electronics)
3. http://en.wikipedia.org/wiki/Schmitt_trigger